Forensic Agent

Implementation

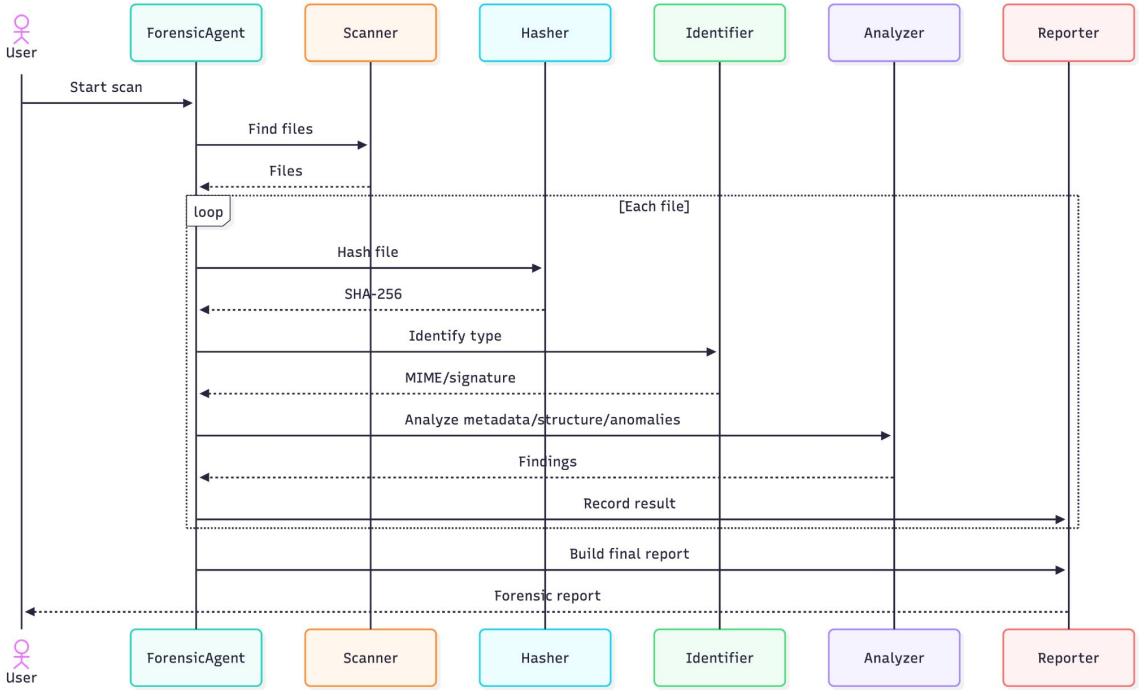
Introduction

The presentation focuses on describing

- The architecture
- The data flow
- Design choices as well as trade-offs

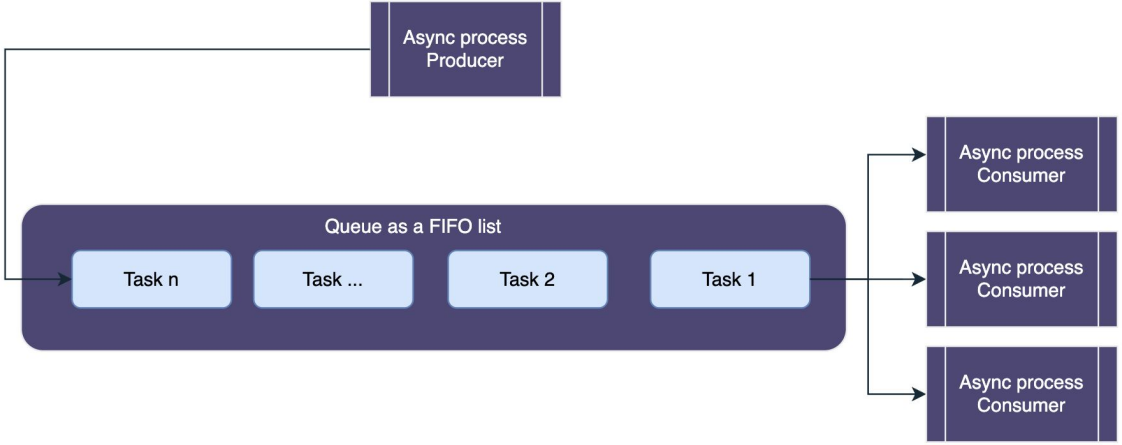
to shape the implementation of a digital forensic agent based on a group proposal titled “**A Prototype Design Report**”

Architecture



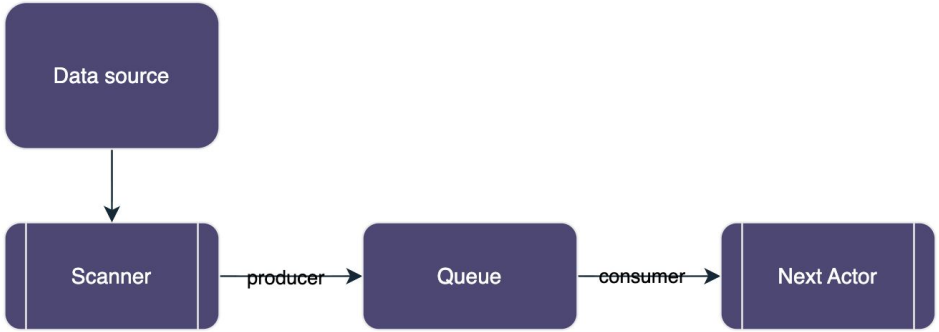
Concurrent Pipeline

When synchronous processes are used, each component plays its part sequentially, from the moment a new file is created until an output is written to the report, resulting in the overall process being blocked from performing its various tasks simultaneously. To mitigate this behaviour, we choose to leverage a concurrent pipeline rather than relying on sequential steps. When a modification occurs inside the watched directory, the file's detail is asynchronously put on a queue where it can be consumed by another process, hence avoiding waiting between each forensic step involved.



Evidence Acquisition

The agent's listener (Scanner) acts as a **sensor** and detects any new file created under the watched directory. The file observer is recursive, which implies the listener will also detect files created in subfolders of the root directory listened to and therefore this directory can be located at any level of the file system. When a new file is found, its path is placed on a Queue, waiting to be picked up by the next actor.



Evidence Integrity

As soon as an evidence is acquired, the agent proceeds on creating hashes from the file content.

By creating a hash of a file under investigation, we record and preserve the integrity of the file and ensure that since it was acquired, the file was not tempered with.

Algorithms used:

- Sha256
- Md5

eg: From the a file `evidence_report.txt`

We get the following:

```
sha256: 5326d465f46f68c350cd37f970b639e9c8d7d053643a44d951d33b601e711c43
```

```
md5: bdb2717e499c2fb9124d529193195c2c
```

Evidence Analysis

We can improve the accuracy in detecting the true nature of a file, not only by reading its extension which could be spoofed, but by extracting its MIME-TYPE, parsing and examining its header.

- The hexadecimal value of PDF header would be `25 50 44 46` and a valid mime-type would be `application/pdf`

- The header for a PNG file is `89 50 4E 47 0D 0A 1A 0A` and its mime-type should `image/png`

When a mismatch between the extension, the mimetype and the header occurs, it is a strong indicator the file is suspicious.

When a new file is detected by the scanner and its extension is either not allowed or shows a mismatch between its mime-type and its header signature, it will be zipped with a text file containing the sha-256 and md5 hashes and copied in an archive folder to be picked up for further investigation.

Reporting

The reporting of files under investigation is logged via two different channels; A comma separated value file and a Sqlite DB file. The report can be downloaded at any time either by downloading the csv file from the server or by connecting to the Sqlite file using a sqlite client.

In order to extend the capabilities of the ArtefactReporter, we choose a behavioural design pattern called the strategy or policy pattern. In the Strategy pattern, an object delegates a specific request to an object that represents a strategy for carrying out the request. An object will only have one state, but it can have many strategies for different requests.

After the agent has run we can observe the containers log by running the following command in the terminal: `docker logs forensic-agent-e2e`

This will display all the operations logged as the agent is performing its forensic tasks.

Reporting

Sample of the report as a csv file

(.venv) christophelaudriec@christophes-MacBook-Pro ForensicAgent % head forensic_report.csv

name,size,path,created_at,modified_at,last_accessed_at,sha256,md5,status

evidence_report.txt,59,/app/watched_directory/evidence_report.txt,2026-04-13T19:30:23.838336,2026-04-13T19:30:23.838336,2026-04-13T19:30:23.920336,5326d465f46f68c350cd37f970b639e9c8d7d053643a44d951d33b601e711c43,bdb2717e499c2fb9124d529193195c2c,clean

evidence_document.pdf,50,/app/watched_directory/evidence_document.pdf,2026-04-13T19:30:24.095336,2026-04-13T19:30:24.095336,2026-04-13T19:30:24.195336,7a6f7637be17a89e0d1a6ebb8ea0ae701b4da00727303ffe16e024a33a73b85c,791e18058cfb343baabd25aaffbc150e,clean

evidence_image.jpg,22,/app/watched_directory/evidence_image.jpg,2026-04-13T19:30:24.607336,2026-04-13T19:30:24.607336,2026-04-13T19:30:24.677336,d20f6ffd523b78a86cd2f916fa34af5d1918d75f7b142237c752ad6b254213ab,57da6f2e0f139e118c7930ae76570491,clean

suspicious_payload.bin,50,/app/watched_directory/suspicious_payload.bin,2026-04-13T19:30:25.113336,2026-04-13T19:30:25.113336,2026-04-13T19:30:25.218336,c05d8e2368558c659d4e226384b03b4a1c576665f0a8bf00751aa286d352df78,af6a1a666c7a8c53d03c7b45c80c350e,suspicious

mystery_data.xyz,46,/app/watched_directory/mystery_data.xyz,2026-04-13T19:30:25.627336,2026-04-13T19:30:25.627336,2026-04-13T19:30:25.695336,b71d19846c9e5d94d53fbd76a149953f2c8fb34c47892e43b0031cf10426f025,0241b4cee53933caa552479f6f3ca32d,suspicious

Full report file from e2e_test_linux available here: https://github.com/christgithub/forensic-agent/blob/main/forensic_report.csv

Unit Testing

Every class's functions are tested at a unit level, ensuring that every functional unit is performing their tasks as expected. For instance, the function `identify_file` from the class **Identifier** is thoroughly tested showing that the value returned is what we expect against a specific use case.
When we run the command `make unit_tests` we get the following output displayed in the console, indicating that the implementation of the identify function belonging to the class Identifier can handle all the use cases it is subjected to.

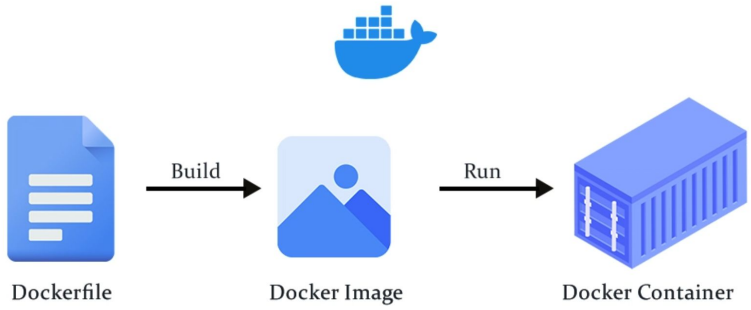
```
ArtefactAnalysis/test_identifier.py::TestIdentifier::test_identify_file_returns_expected_structure PASSED
ArtefactAnalysis/test_identifier.py::TestIdentifier::test_identify_file_valid_allowed_extension PASSED
ArtefactAnalysis/test_identifier.py::TestIdentifier::test_identify_file_spoofed_file_mime_mismatch PASSED
ArtefactAnalysis/test_identifier.py::TestIdentifier::test_identify_file_unknown_extension_is_invalid PASSED
ArtefactAnalysis/test_identifier.py::TestIdentifier::test_identify_file_no_extension_is_invalid PASSED ...
```

Example

`ArtefactAnalysis/test_identifier.py::TestIdentifier::test_identify_file_unknown_extension_is_invalid` validates that the function `identify_file` returns the right value when it is passed an unknown extension. We keep the allowed extensions in a constant dictionary under the package domain and the extension tested `.rar` is not included.

End to End Testing

The end to end test demonstrates how the forensic agent proceeds in generating a report of any files created in a folder under scrutiny in a specific environment. We leverage Docker containers to run our agent on a specific operating system.



The Makefile contains a key with value `**e2e_tests**` and by running `make e2e_tests` in the application terminal we start the agent end to end tests and we can see the agent processing new files under the watched folder and logging all its operations to the container and outputting its actions to the terminal.

End to End Testing

If we navigate to the application’s root directory once the tests have terminated, and key in the commands:

- `docker cp forensic-agent-e2e:/app/forensic_report.csv .`
- `docker cp forensic-agent-e2e:/app/forensic_report.db .`

We can expect to see 2 files copied locally from the running container: forensic_report.csv and forensic_report.db

Those files contain the report output for all the use cases described and tested in e2e in 2 different formats.

Considerations and Improvements

Scaling is probably one of the most prominent issues with the design of the agent. A more robust pipeline levering parallelism over multiple cores would enhance the scalability of the agent under heavier load. For larger systems, implementing a distributed multi-agent approach would be more adapted.

If the investigated system is a node on a network, sending the report via encrypted payloads would allow keeping the reports safely on distributed servers as it was suggested on.

The analysis of the files under investigation was kept simple for this exercise, but a deeper and thorough analysis of the file content, such as steganography detection would strengthen the validity of the result. The objective of steganography is to embed a distinct secret message into a cover object which could be anything to be transferred over the internet such as images, text, audio, PDF files, videos etc (Dalal and Juneja, 2021), therefore, enforcing this type of technique would enhance the veracity of our investigation.